Chapter 5 — Exploring Categorical Variables

Exploratory Data Analysis (EDA) · Part II: Univariate Analysis

Purpose: Investigate categories using counts, proportions, frequency distributions, cardinality, rare-category analysis, and visual summaries. Learn to interpret labels and ordering without confusing frequency with importance.

Learning objectives

• Distinguish nominal, ordinal, binary, and identifier variables.

• Build frequency tables and calculate proportions and percentages.

• Interpret cardinality, dominance, imbalance, and rare categories.

• Handle missing categories explicitly and choose the correct denominator.

• Create and interpret categorical bar charts.

• Build a reusable categorical profile in Pandas.

• Recognize label inconsistencies and limitations in category interpretation.

1. Review of earlier concepts

Chapter 1 — What Is EDA?

EDA systematically investigates data structure, quality, patterns, and limitations. Ask: What data do I have? Can I trust it? What patterns or relationships does it contain? Quantitative and visual EDA should support one another.

Chapter 2 — Understanding the Dataset

Establish what one row represents, what each column measures, and the dataset's granularity. A category label is interpretable only when its meaning and context are known.

Review of Chapter 4 — Exploring Numerical Variables

Chapter 4 examined numerical measurements through counts, mean, median, mode, spread, quantiles, skewness, histograms, boxplots, and ECDFs. The central lesson: combine quantitative summaries with visual evidence and investigate unusual values rather than automatically deleting them.

Discussion of Chapter 4 assignment

For [12, 14, 14, 15, 18, 20, 21, 22, 24, 60], count = 10, mean = 22, median = 19, mode = 14, minimum = 12, maximum = 60, range = 48. With Pandas' default linear quantile interpolation, Q1 = 14.25, Q3 = 21.75, IQR = 7.5. Sample variance = 194 and sample standard deviation ≈ 13.93. The value 60 pulls the mean upward and increases the range and standard deviation.

A strong practical submission identifies measurements rather than IDs, reports missingness, calculates summaries, uses complementary plots, and separates an unusual observation from a hypothesis about its cause.

What the Chapter 4 coding project should teach you

The Numerical Variable Profiler should reproducibly calculate summaries, draw labeled histogram/boxplot/ECDF views, flag observations for investigation, and explain limitations. We apply the same discipline to categories.

2. Preview of this chapter

An Occupation column might contain Teacher, Engineer, Student, and Business Owner. A few rows cannot reveal which occupations are common, whether one category dominates, whether labels are inconsistent, or whether rare categories need investigation. We use frequency tables, proportions, cardinality, and plots.

3. What is a categorical variable?

A categorical variable assigns each observation to a group or label. Its values represent membership rather than numerical amounts. Examples include occupation, city, product type, payment method, department, and satisfaction level.

Nominal variables

Nominal categories have no inherent order: city, occupation, blood group, product category, payment method. Alphabetical order is convenient for display but does not create a ranking.

Ordinal variables

Ordinal categories have a meaningful order, but distances between adjacent categories are not necessarily equal: low/medium/high risk, education level, or dissatisfied/neutral/satisfied.

Binary variables

Binary variables have two meaningful categories, such as yes/no or active/inactive. They may be stored as Boolean, integer, or text.

Identifiers

Customer IDs and postal codes may contain digits but identify entities or locations; they are not ordinary quantitative measurements.

Variable	Example values	Type
City	Dhaka, Sylhet, Khulna	Nominal
Satisfaction	Low, Medium, High	Ordinal
Purchased	Yes, No	Binary
Customer ID	C1001, C1002	Identifier

Important: Ordinal categories have order, but do not assume equal distances between them. Coding Low=1, Medium=2, High=3 does not prove equal spacing.

4. Establish meaning and quality

Before calculating frequencies, inspect the definition, allowed values, missingness, and label consistency. These checks connect categorical EDA with Chapter 3's data-quality audit.

```
df = pd.DataFrame({
    "Occupation": [
        "Teacher", "Engineer", "Student", "Teacher",
        "teacher", "Engineer ", "Business Owner", None,
        "Student", "Teacher", "Other", "Engineer"
    ],
    "Satisfaction": [
        "High", "Medium", "High", "Low", "Medium",
        "High", "Low", "Medium", "High", "Medium",
        "Low", "High"
    ]
})

occupation = df["Occupation"]
print(df.shape)
print(df.dtypes)
print(occupation.head(10))
print("Missing:", occupation.isna().sum())
print("Unique:", occupation.nunique())
```

Inconsistent labels

Teacher and teacher may mean the same thing; Engineer and Engineer with a trailing space may also match. These are potential inconsistencies to investigate. Do not silently merge labels without checking their source and meaning.

```
print(occupation.value_counts(dropna=False))
print(sorted(occupation.dropna().unique()))
```

The first command includes missing values. The second displays distinct non-missing labels. For high-cardinality variables, inspect frequent values and likely near-duplicates instead of printing a huge list.

Missing is not automatically a category

A missing value means a category was not observed or recorded. It is not automatically Unknown, Other, or Not Applicable. These labels have distinct meanings: Unknown means the category is not known; Not recorded describes collection failure; Not applicable means the variable does not apply; Other is a residual group.

5. Frequency tables

A frequency table counts how often each category occurs.

Absolute frequency

Absolute frequency is the number of observations in a category. value_counts() sorts by descending frequency by default and excludes missing values unless dropna=False.

```
occupation = df["Occupation"]
counts = occupation.value_counts()
print(counts)

counts_including_missing = occupation.value_counts(
    dropna=False
)
print(counts_including_missing)
```

Relative frequency and percentage

Relative frequency = category count / number of observations in the denominator. Percentage = relative frequency × 100. With the default missing-value exclusion, the denominator is the number of non-missing observations.

```
proportions = occupation.value_counts(normalize=True)
percentages = 100 * occupation.value_counts(normalize=True)
print(proportions)
print(percentages)

all_proportions = occupation.value_counts(
    normalize=True, dropna=False
)
print(all_proportions)
```

The denominator changes when missing values are included. State which population your percentages describe.

Build a frequency table explicitly

```
freq = occupation.value_counts(dropna=False).rename("count")
freq_table = freq.to_frame()
freq_table["proportion"] = freq_table["count"] / len(occupation)
freq_table["percentage"] = 100 * freq_table["proportion"]
print(freq_table)
```

This table uses all rows—including missing values—in the denominator. Choose the denominator to match the analytical question.

6. Cardinality, dominance, and rare categories

Cardinality

Cardinality is the number of distinct categories. A city column with 8 values has lower cardinality than a product-code column with 20,000 values. High cardinality is not automatically a quality problem; it may reflect real detail, or indicate an identifier/free-text field.

```
print("Categories:", occupation.nunique())
print("Including missing:", occupation.nunique(dropna=False))
```

Dominance and imbalance

A distribution is dominated when a small number of categories account for a large share of observations. This may accurately reflect the population or collection process.

```
shares = occupation.value_counts(normalize=True)
print("Largest share:", shares.max())
print("Top category:", shares.index[0])
```

The most frequent category is not necessarily the most important, valuable, or deserving of attention.

Rare categories

Rare categories have low counts or proportions. They may be genuine uncommon cases, data-entry errors, overly detailed labels, or categories requiring domain investigation.

```
counts = occupation.value_counts()
rare = counts[counts < 2]
print(rare)
```

Fewer than two is only an illustrative threshold. There is no universal definition of rare; choose a threshold based on sample size, purpose, and domain context.

Top-k summaries

```
top_categories = occupation.value_counts().head(5)
print(top_categories)
```

Disclose when a table is truncated to the top categories. Do not imply that the displayed values represent all observations.

7. Ordered categorical variables

An ordinal variable's natural order may differ from alphabetical or frequency order. Satisfaction should usually appear as Low, Medium, High.

```
satisfaction_order = ["Low", "Medium", "High"]

df["Satisfaction"] = pd.Categorical(
    df["Satisfaction"],
    categories=satisfaction_order,
    ordered=True
)

print(df["Satisfaction"].value_counts(sort=False))
```

Declaring categories makes the intended order explicit. Values outside the declared categories may become missing during conversion, so inspect unexpected labels first.

• For nominal variables, compare counts and proportions without inventing a ranking.

• For ordinal variables, preserve meaningful order and consider cumulative proportions.

• Do not assume ordinal steps are equally spaced without evidence.

8. Visualizing categorical variables

Bar charts are the standard starting point. They display counts or proportions for distinct categories. Unlike histograms, bars normally have gaps because categories are separate.

Count bar chart

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.countplot(
    data=df,
    x="Occupation",
    order=df["Occupation"].value_counts().index,
    ax=ax
)
ax.set_title("Customer Occupation Counts")
ax.set_xlabel("Occupation")
ax.set_ylabel("Number of customers")
ax.tick_params(axis="x", rotation=35)
fig.tight_layout()
plt.show()
```

The order argument sorts by observed frequency. For many categories or long labels, a horizontal chart is often easier to read.

Horizontal bar chart

```
counts = df["Occupation"].value_counts()
fig, ax = plt.subplots(figsize=(8, 5))
ax.barh(counts.index.astype(str), counts.values)
ax.set_title("Customer Occupation Counts")
ax.set_xlabel("Number of customers")
ax.set_ylabel("Occupation")
ax.invert_yaxis()
fig.tight_layout()
plt.show()
```

invert_yaxis() places the most frequent category at the top when counts are sorted descending.

Proportion bar chart

```
shares = df["Occupation"].value_counts(normalize=True)
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(shares.index.astype(str), shares.values)
ax.set_title("Occupation Distribution")
ax.set_xlabel("Occupation")
ax.set_ylabel("Proportion")
ax.tick_params(axis="x", rotation=35)
ax.set_ylim(0, 1)
fig.tight_layout()
plt.show()
```

Proportions help compare datasets or groups of different sizes. Label the axis so readers do not confuse proportions with counts.

Pie charts

A pie chart can work for a few mutually exclusive categories that form one whole. Comparing similar slices is difficult, and many categories create clutter. A sorted bar chart is usually easier.

Chart	Useful for	Caution
Vertical bar	Few/moderate categories	Long labels can collide
Horizontal bar	Many categories or long labels	Keep order meaningful
Proportion bar	Comparing shares	Label proportion/percent
Pie chart	A few parts of one whole	Hard to compare slices

9. A reusable categorical summary

This function returns a frequency table and a compact variable-level summary. It expresses each category as a share of all rows, including missing values in the denominator.

```
def summarize_categorical(series: pd.Series):
    total = len(series)
    missing = int(series.isna().sum())
    counts = series.value_counts(dropna=False)

    result = counts.rename("count").to_frame()
    result["proportion_all_rows"] = result["count"] / total
    result["percentage_all_rows"] = (
        100 * result["proportion_all_rows"]
    )

    summary = pd.Series({
        "total_rows": total,
        "missing_count": missing,
        "missing_percentage": 100 * missing / total if total else 0,
        "unique_non_missing": series.nunique(),
        "unique_including_missing": series.nunique(
            dropna=False
        ),
    })
    return result, summary

frequency, summary = summarize_categorical(df["Occupation"])
print(frequency)
print(summary)
```

If you want proportions among observed categories only, use value_counts(normalize=True) with the default missing-value exclusion. Always state the denominator.

10. First look at relationships: cross-tabulation

A contingency table (crosstab) counts combinations of categories in two categorical variables.

```
pd.crosstab(
    df["Occupation"],
    df["Satisfaction"],
    margins=True
)
```

Raw counts can mislead when group sizes differ. Row-normalized proportions answer: within each occupation, how are satisfaction responses distributed?

```
pd.crosstab(
    df["Occupation"],
    df["Satisfaction"],
    normalize="index"
)
```

These tables are descriptive and do not establish causation. We will study categorical relationships and grouped analysis in later chapters.

11. Common mistakes

• Treating nominal categories as if they have a meaningful order.

• Assuming ordinal categories are equally spaced.

• Confusing missing values with Unknown, Other, or Not Applicable.

• Reporting percentages without explaining the denominator.

• Showing only top categories without disclosing truncation.

• Treating rare categories as errors without checking context.

• Using pie charts with too many slices.

• Comparing raw counts across groups of very different sizes.

• Assuming the most frequent category is the most important.

• Silently merging similar-looking labels that may mean different things.

12. Chapter summary

• Categorical variables represent labels or group membership, not numerical amounts.

• Nominal categories have no inherent order; ordinal categories do.

• Frequency tables provide counts, proportions, and percentages.

• Cardinality is the number of distinct categories.

• Dominance and rare categories describe frequency patterns.

• Missing values require explicit handling and a clear denominator.

• Bar charts are usually the clearest categorical visualization.

• Crosstabs provide a first descriptive view of categorical relationships.

13. Assignment

Part A — Conceptual questions

• Explain nominal, ordinal, and binary variables, giving two examples of each.

• Why is Customer ID generally unsuitable for ordinary frequency analysis?

• Explain absolute versus relative frequency.

• What is cardinality, and why might high cardinality deserve investigation?

• Why should a rare category not automatically be removed or grouped?

• Why must the denominator be stated when reporting percentages?

• When is a horizontal bar chart preferable?

• Why does an ordinal variable need an explicit category order?

Part B — Quantitative practice

Consider: [A, B, A, C, B, A, D, B, A, C, B, A, None, D, B, A, C, B, A, A]. Construct a frequency table with counts, proportions, and percentages. Include missingness as a separate row. Calculate category shares among all rows and among non-missing observations.

Part C — Practical EDA

Use a dataset containing at least two categorical variables. For each selected variable:

• Document its meaning and whether it is nominal, ordinal, or binary.

• Report total rows, missing count, unique category count, and a frequency table.

• Report counts and percentages, stating the denominator.

• Identify the most common and rare categories.

• Create a readable bar chart; use horizontal layout for long labels.

• For an ordinal variable, declare its meaningful order and check unexpected labels.

• Write an interpretation and identify at least one limitation.

Part D — Cross-tabulation

Choose two categorical variables and create a contingency table and a row-normalized version. Explain what each reveals and why raw counts and within-row proportions answer different questions.

Part E — Reflection

Choose an ambiguous label such as Other, Unknown, or Miscellaneous. Explain what metadata or domain knowledge you need before interpreting it.

14. Coding project — Categorical Variable Profiler

Extend the cumulative EDA master notebook with a reusable categorical profiling section. Keep it consistent with the numerical profiler from Chapter 4.

Requirements

• Load a dataset and identify at least three meaningful categorical variables.

• Document each variable's meaning and type: nominal, ordinal, or binary.

• Create a reusable function reporting total rows, missing count/percentage, unique categories, counts, proportions, and percentages.

• Make the denominator explicit and optionally include missingness as a displayed category.

• Create sorted frequency tables and top-k summaries for high-cardinality variables.

• Identify rare categories using a documented threshold; do not automatically merge or remove them.

• Create at least three labeled bar charts, including a horizontal chart where useful.

• For an ordinal variable, define its category order and check unexpected labels.

• Create one crosstab and one row-normalized crosstab.

• Write findings that distinguish observed frequencies from explanations or causal claims.

Deliverables

• Updated cumulative EDA master notebook—not a disconnected notebook.

• Reusable categorical profiling function.

• Frequency tables with clearly defined denominators.

• At least three labeled categorical plots.

• One contingency table and one normalized contingency table.

• A findings section documenting assumptions, missingness, rare categories, and limitations.

Success criterion: Another analyst can rerun the notebook, understand how every category was counted, and see exactly how missing values and denominators were handled.

Preview of Next Chapter

Chapter 6 — Exploring Date and Time Variables

Next, we will investigate date and time data: parsing dates, understanding timestamps and time zones, extracting calendar components, examining durations and intervals, identifying temporal gaps, and recognizing patterns such as seasonality and day-of-week effects. We will distinguish genuine temporal variables from strings that merely look like dates.